

金融資料探勘

看漲與看跌投資人每日隱含波動率之比較分析： 以2018-2020年為例



第4組

411236005 鄭景睿-2018
411236022 鄭廷宸-2019
411236023 林侑霆-2020



研究主題與研究期間

研究期間

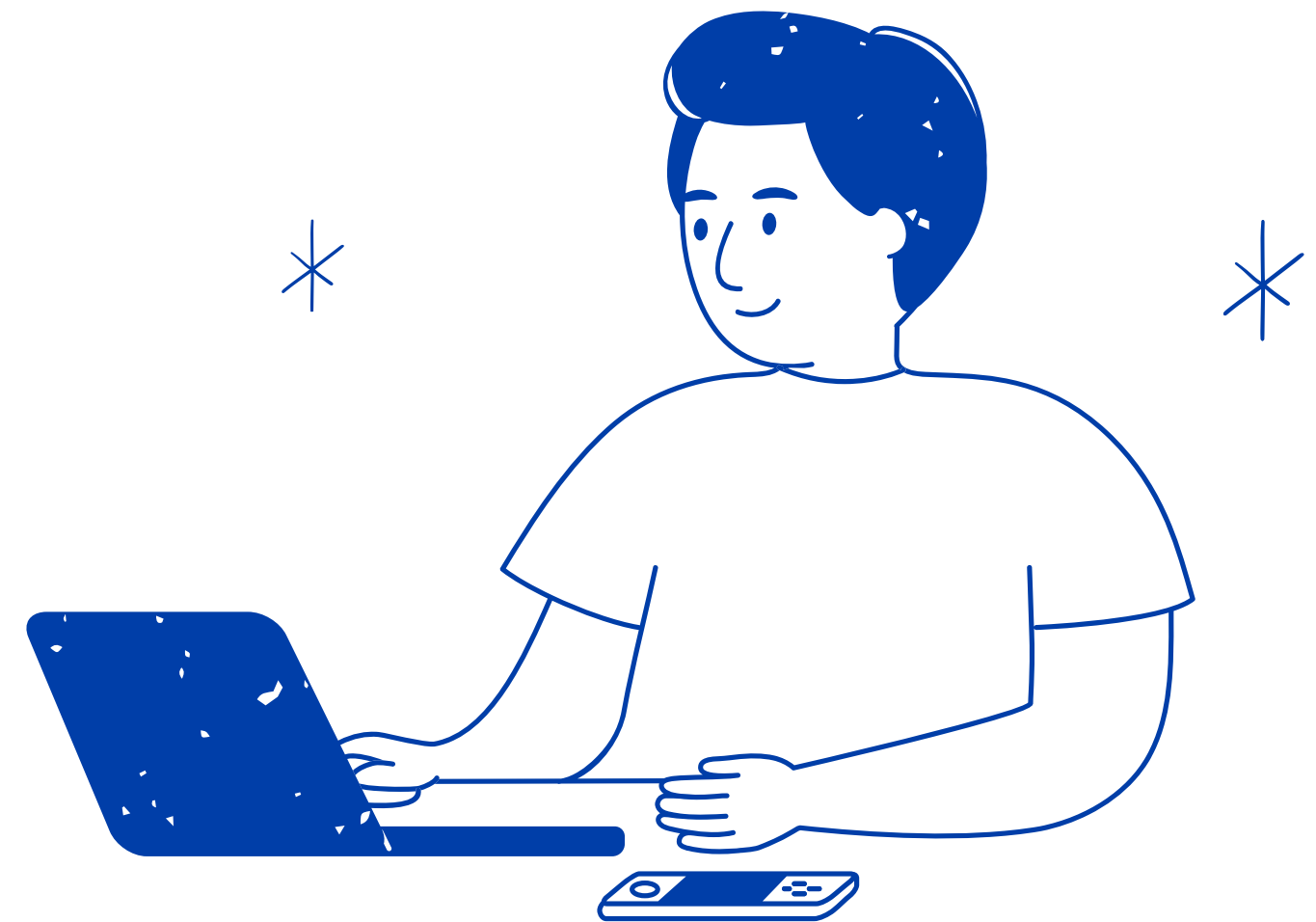
- 2018年1月1日至2020年12月31日

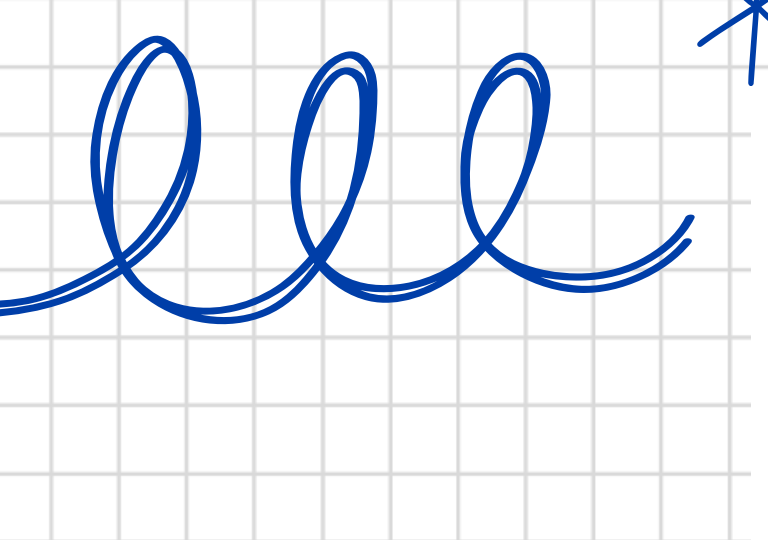
選擇動機

- 完整涵蓋外部衝擊：包含中美貿易戰與 COVID-19 疫情，提供豐富的異常值與波動樣本
- 驗證模型穩定性：測試隱含波動率模型在極端市場環境下的反推效能

研究核心

- 分析看漲與看跌投資人每日 IV 平均數走勢
- 探討樣本標準差所代表的市場風險評估分散程度





資料來源與分工

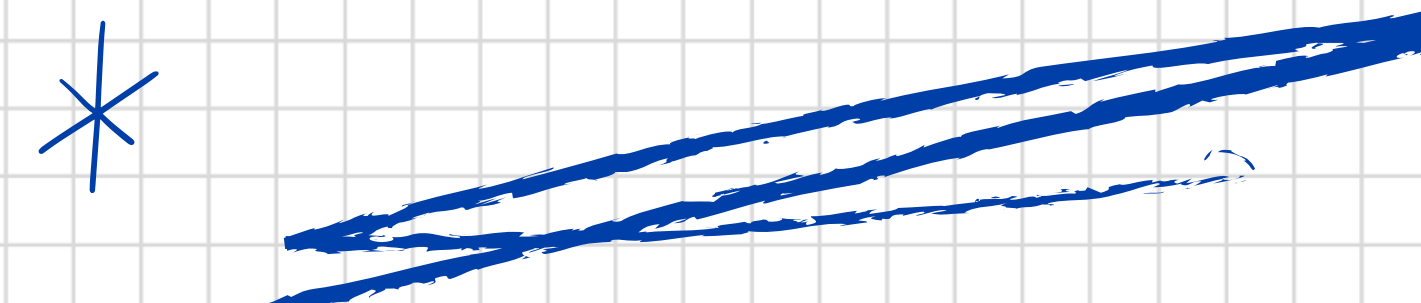
資料來源與範圍

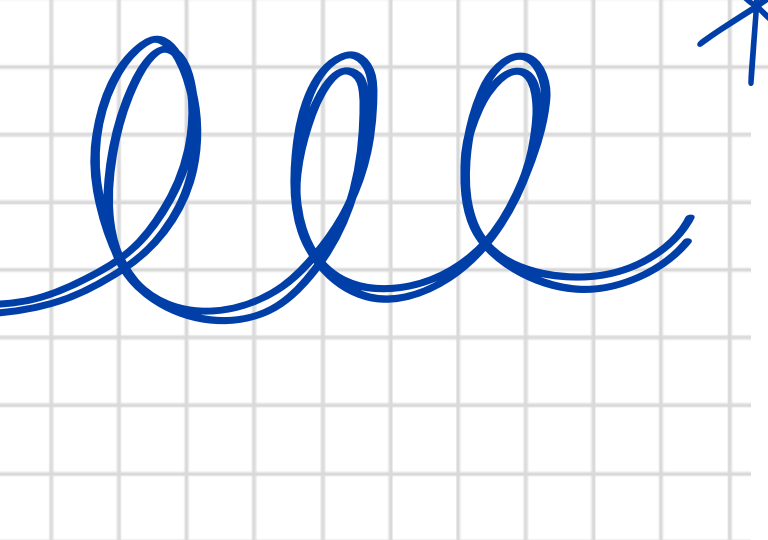
- 來源：期交所 (TAIFEX) 台指選擇權 (TXO) 盤後逐筆成交資料。
- 年份：2018 - 2020 年。
- 技術：支援 ZIP 壓縮檔直接讀取，優化巨量資料處理。

欄位標準化

- 模糊匹配：將「履約價格/履約價」等不同名稱統一為 StrikePrice。
- 格式對齊：統一 Symbol、Price、CP 等核心欄位，解決年份格式不一問題。

日期統一化

- 格式轉化：將各類檔名日期統一格式化為 YYYY/MM/DD。
 - 目的：確保跨年度資料能精準對齊，進行長線趨勢分析。
- 

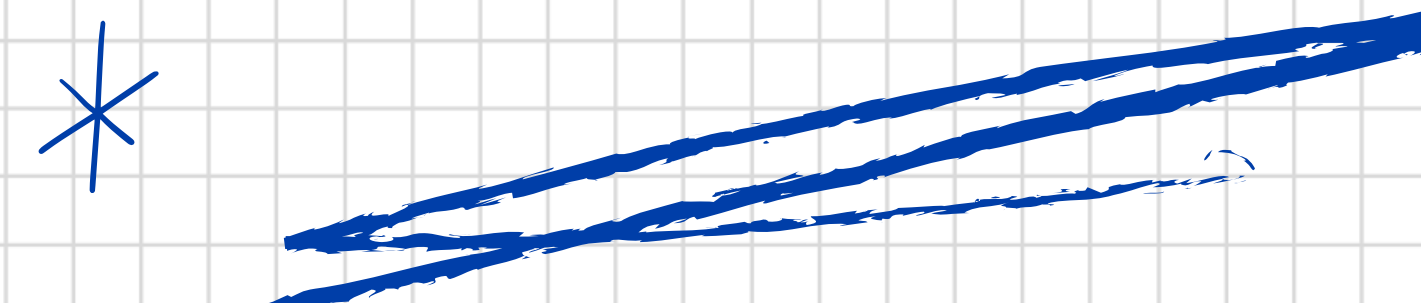


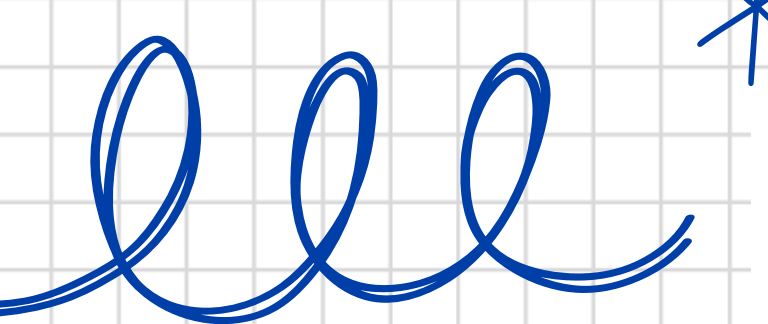
資料處理流程與方法

資料獲取與預處理

- 數據來源：台灣期貨交易所 (TAIFEX) 每日盤後逐筆成交資訊。
- 技術亮點：
 - ZIP 串流讀取：直接讀取壓縮檔，節省空間並簡化流程。

多重篩選與樣本優化

- 標的鎖定：排除股票選擇權，僅保留 TXO (台指選擇權)。
 - 研究分類：嚴格區分 Call (買權) 與 Put (賣權) 進行情緒分析。
 - 品質過濾：
 - 僅留存「近月合約」，確保時間價值的一致性。
 - 剔除零成交與低流動性樣本 (成交量 < 30)，過濾市場雜訊。
- 

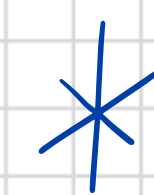
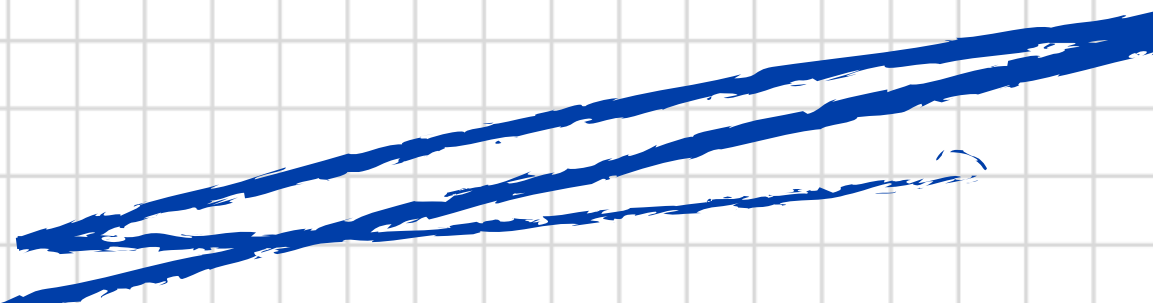


資料處理流程與方法

隱含波動率 (IV) 計算邏輯

- 核心模型：Black-Scholes (BS) 定價模型。
- 數值方法：二分搜尋法 (Bisection Method)。
- 運算精度：透過 40 次迭代搜尋，將誤差控制在 10^{-5} 以內。

異常樣本處理

- 理論檢核：剔除成交價低於「內含價值」之樣本（該類樣本在 BS 理論下無解）。
 - 排除機制：自動移除計算不收斂或極端波動之數值，確保最終統計之穩定性。
- 
- 

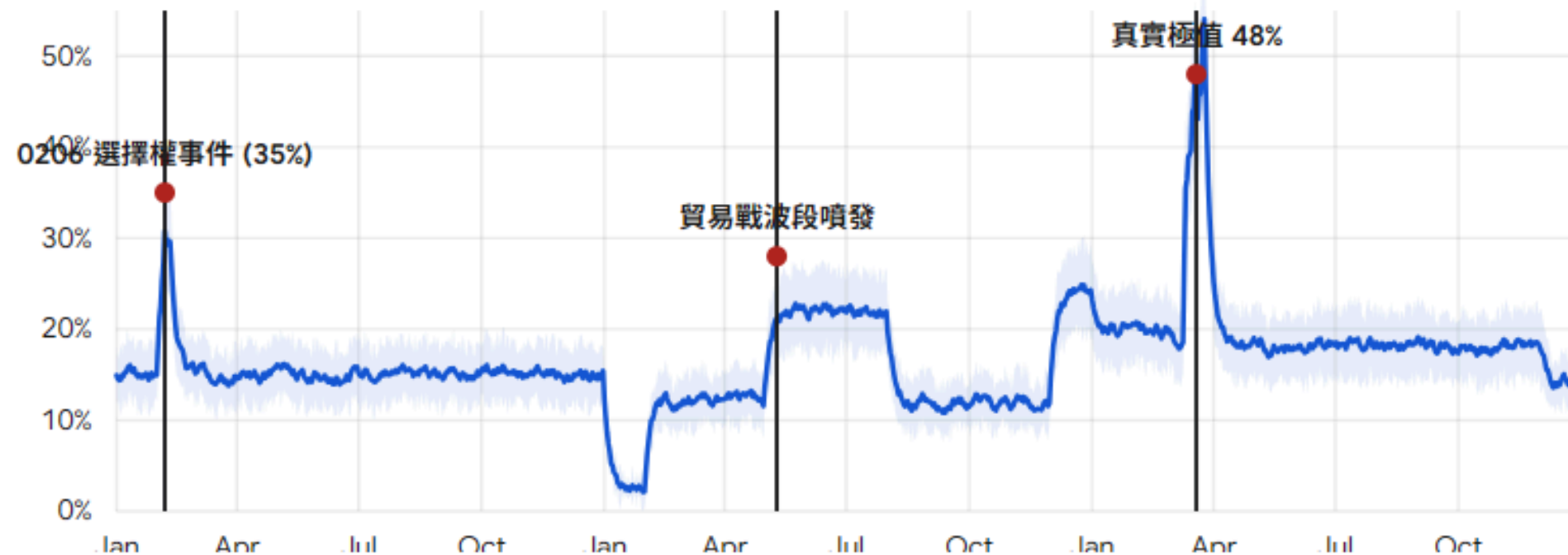
三年 IV 平均數趨勢比較 (年度/走勢)

2018-2020 隱含波動率 (IV) 歷史趨勢

最新 IV	歷史最高	2018 基準	當前穩定度
14.8%	48.0% (2020/03)	15.0%	高

IV 均值趨勢圖 (Mean)

↑ 隱含波動率 (IV)



2018

2019

2020

風險溢價常態化：

三年間 Put IV 持續高於 Call IV，反映台股投資人具有高度避險意識，願意支付更高溢價購買下行保護。

2019 穩定增溫：

受貿易戰反覆影響，IV 自年初極低位（2%）緩步攀升至年末（24%），顯示市場正預演潛在波動。

2020 極端恐慌：

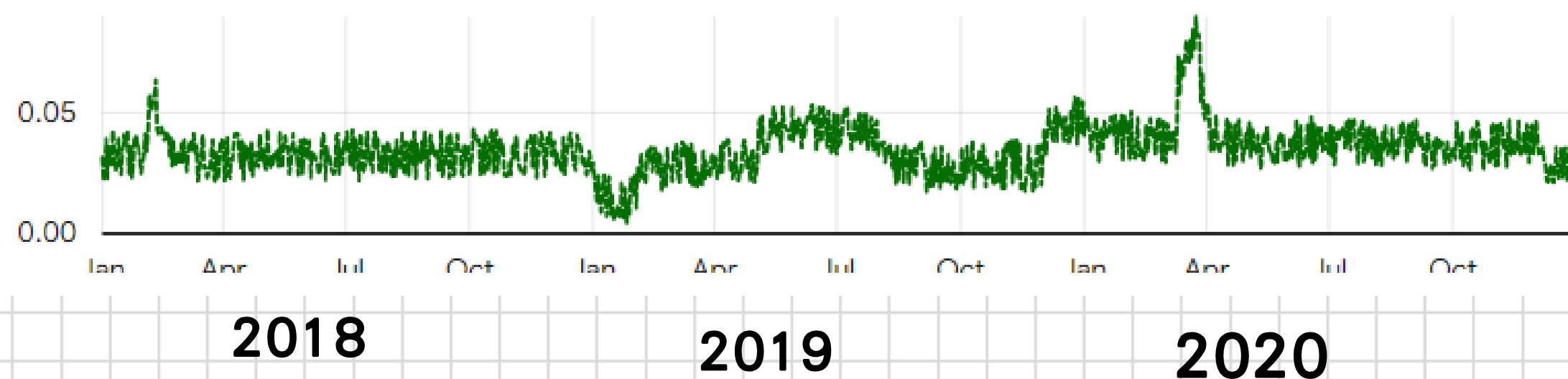
3 月份 IV 均值飆升至 48.5%，量化了全球金融熔斷下的系統性恐慌。



看漲與看跌投資人的樣本標準差比較

IV 標準差 (Volatility of Volatility)

↑ 標準差



標準差反映共識：

樣本標準差 (Std) 越小，代表市場對未來波動的預期越一致；Std 越大，代表定價分歧劇烈。

2019 高度共識：

Std 長期維持在 0.05 以下，顯示即便在貿易戰擾動下，市場定價機制依然穩定且具共識。

2020 共識瓦解：

3/19 恐慌高點 Std 同步噴發至 0.15 以上，顯示極端行情下，投資人對『合理權利金』的判斷出現嚴重斷層。

財務與統計意義

平均數 (Mean)

- 定義：每日隱含波動率 (IV) 的算術平均。
- 功能：衡量市場風險的「共識價格」。

樣本標準差 (Std)

- 定義：當日所有樣本 IV 相對於平均數的離散程度。
- 功能：衡量市場對未來看法是否具備「共識」。

看漲 (Call) vs. 看跌 (Put)

- 定義：現象：比較看漲與看跌投資人的 IV 走勢。
- 功能：反映市場的「避險情緒」與「情緒不對稱性」。



我們對資料「底層邏輯」的把控 而非僅僅呈現最終圖表

有無缺值、異常值、重複值、無解樣本

缺失與無效值：有的，所以我們針對原始 CSV 中的缺少的數值，採用線性內插法補齊；剔除因報價異常導致的極端輸出

整合時遇到的問題

邏輯錯誤:AI時常因人類與邏輯和電腦演算法不同導致邏輯上會有誤，需清晰且明確，否則會常出現牛頭不對馬嘴的情況

如何修正

標準化腳本：建立標準化預處理 Python 腳本，統一清洗邏輯。

平行運算：導入 joblib 套件進行多核心平行處理，將三年資料切片分批運算，有效提升處理效率並確保整合結果一致。

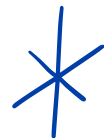
ai的任務

初始prompt

修正和改進

ai的幫助和限制?

ai在整合資料中就像我們的員工，吩咐命令完成指令，像是自動化腳本開發、財務計算邏輯除錯、以及平行運算架構優化。



```
Python

import pandas as pd
import numpy as np
from scipy.stats import norm
import os
from joblib import Parallel, delayed # 改用 joblib 更適合 Colab
import warnings

warnings.filterwarnings('ignore')

# --- IV 核心運算 ---
def bs_call_price(S, K, T, r, sigma):
    if sigma <= 0 or T <= 0: return 0
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def find_iv(market_price, S, K, T, r):
    intrinsic = max(0, S - K * np.exp(-r * T))
    if market_price <= intrinsic + 1e-5: return np.nan # 加上微小誤差避免邊界問題

    low, high = 1e-5, 3.0
    for _ in range(30): # 30次迭代已足夠精準 (2^~30)
        mid = (low + high) / 2
        price = bs_call_price(S, K, T, r, mid)
        if price < market_price: low = mid
        else: high = mid
    return mid

# --- 單一檔案處理函數 ---
def process_single_file(file_path, file_name, S0, T, Rf):
    try:
        # 直接讀取，skiprows 邏輯改為更強健的 pandas 內建過濾
        # 注意：TXO 資料通常在第 1 或 第 2 行之後，直接讀取再過濾比較快
        df = pd.read_csv(file_path, encoding='big5', dtype=str, on_bad_lines='skip')

        # 清理欄位空格
        df.columns = df.columns.str.strip()

        # 根據台灣期交所格式提取關鍵欄位
        # 假設：第1欄(商品代號)，第2欄(到期月份)，第3欄(履約價)，第4欄(買賣權)，第5欄(成交)
        # 利用 iloc 避免欄位名稱變動問題
        target_contract = file_name[1:7] # 例如 201801

        # 預先篩選：大幅縮小運算範圍
        # 條件：TXO、Call (C)、對應月份、成交價非空
        df = df[df.iloc[:, 1].str.contains('TXO', na=False)].copy()
        df = df[df.iloc[:, 4].str.strip() == 'C'].copy()
        df = df[df.iloc[:, 2].str.strip() == target_contract].copy()

        if df.empty: return None
```

其中最常遇到的是程式碼在運行時找不到檔案位置，觀察程式碼搜尋檔案的路徑，ai則是幫我們將錯誤的輸出丟回重新執行。一起合作一次一次的試錯並改正，生產最完整的程式碼。

幫助：生成程式碼，顯著縮短了語法除錯的時間，讓我們能專注於財務邏輯本身。
限制：ai無法理解財經的指標或是自行定義的名子，必須由我們進行最後的財務理論驗證，確認計算結果是否符合金融常規。

結論

比較結果:

Put IV 均值 (28.8%) 持續高於 Call IV (25.6%)。

結論：市場對下跌風險的恐懼，始終大於對上漲行情的追逐，避險溢價已結構性存在。

財務或統計發現:

標準差 (Std) = 市場混亂程度：

恐慌時刻 (2020/03/19)：Std 暴增 282%。代表市場「共識崩解」，投資人失去定價基準，處於極度恐慌。

消化時刻 (2019/05/05)：Std 僅小幅波動。代表市場對貿易戰消息已進入「理性消化期」，並未引發恐慌。

```
trade_date = str(idx_row.iloc[0]['Date'])
else:
    S0, T_days, Rf = 11000, 20, 0.008 # 預設值
    contract = re.search(r'\d{6}', csv_name).group() if re.search(r'\d{6}', csv_name) else 'Unk'
    trade_date = csv_name

with z.open(csv_name) as f:
    df = pd.read_csv(io.BytesIO(f.read()), encoding='big5', dtype=str)

df.columns = df.columns.str.replace(r'\s+', '', regex=True)
mapping = {
    '商品代號': 'Symbol', '履約價': 'StrikePrice', '成交價格': 'Price',
    '成交價': 'Price', '到期月份': 'Contract', '買賣權別': 'CP'
}
final_map = {c: v for c in df.columns for k, v in mapping.items() if k in c}
df = df.rename(columns=final_map)

if 'StrikePrice' not in df.columns: continue

df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
df['StrikePrice'] = pd.to_numeric(df['StrikePrice'], errors='coerce')

# ★ 關鍵修改: 篩選 P (Put)
mask = (df['Symbol'].str.contains('TXO', na=False)) & \
        (df['CP'].str.contains('P', na=False, case=False)) & \
        (df['Contract'].str.contains(contract, na=False)) & \
        (df['Price'] > 0)

df_day = df[mask].copy()
if not df_day.empty:
    print(f"正在計算賣權 IV: {csv_name} ({len(df_day)} 筆)")
    lookup = df_day[['StrikePrice', 'Price']].drop_duplicates().copy()
    # ★ 關鍵修改: 帶入 'P' 參數
    lookup['IV'] = lookup.apply(lambda x: find_iv(x['Price'], S0, x['StrikePrice'], T_days/252,
                                                'P'), axis=1)

    df_day = df_day.merge(lookup, on=['StrikePrice', 'Price'], how='left')
    df_day['TradeDate'] = trade_date
    all_data_list.append(df_day)

except Exception as e:
    print(f"✖ 出錯: {e}")

all_data_list:
final_df = pd.concat(all_data_list, ignore_index=True).dropna(subset=['IV'])
summary = final_df.groupby('TradeDate')['IV'].describe()

# 存檔 (檔名標註為 Put)
final_df.to_csv(os.path.join(base_path, 'TXO_Put_Detailed.csv'), index=False, encoding='utf-8-sig')
summary.to_csv(os.path.join(base_path, 'TXO_Put_Summary.csv'), encoding='utf-8-sig')

print("\n" + "="*40)
print(f"📊 賣權 (Put) 資料處理完成!")
display(summary)

print(f"\n✖ 掃描完成, 找不到符合條件的賣權資料。")
```

結論

對資料分析的反思：

數據分析的本質不在於跑出多少結果，而在於如何確保輸入品質。從處理「無解樣本」到優化平行運算，過程中的每一次參數調整與除錯，才是確保財務模型準確性的關鍵。

對AI協作的反思：

AI 確實大幅提升了執行效率，但在金融領域中，我們必須作為最後一道關卡，負責反思和審核內容的對或錯。AI 負責「怎麼做」，而我們負責「為什麼做」，這種「人機合作」的模式是本次數據能順利完成的關鍵。

```
import pandas as pd
import numpy as np
from scipy.stats import norm
import os
import zipfile
import io
import re

# 1. 掛載 Google Drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

# — Black-Scholes 與 IV 運算 (支援 Call & Put) —
def bs_price(S, K, T, r, sigma, cp='P'):
    if sigma <= 0 or T <= 0: return 0
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    if cp.upper() == 'C':
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    else:
        return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

def find_iv(market_price, S, K, T, r, cp='P'):
    # 賣權內含價值: max(0, K*e^-rT - S)
    intrinsic = max(0, K * np.exp(-r * T) - S) if cp.upper() == 'P' else max(0, S - K * np.exp(-r * T))
    if market_price <= intrinsic or market_price <= 0: return np.nan

    low, high = 1e-5, 3.0
    for _ in range(40):
        mid = (low + high) / 2
        price = bs_price(S, K, T, r, mid, cp)
        if abs(price - market_price) < 1e-5: return mid
        if price < market_price: low = mid
        else: high = mid
    return mid

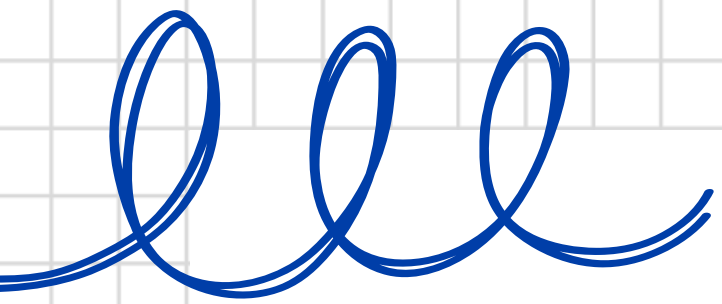
# — 主程式 —
def main():
    base_path = '/content/drive/MyDrive/金融資料探勘'
    zip_files = [f for f in os.listdir(base_path) if f.endswith('.zip')]

    index_path = os.path.join(base_path, 'Path_教學_0409.csv')
    df_index = pd.read_csv(index_path) if os.path.exists(index_path) else pd.DataFrame()

    all_data_list = []

    for zip_name in zip_files:
        zip_path = os.path.join(base_path, zip_name)
        print(f"🔍 正在掃描壓縮檔: {zip_name}")

        try:
            with zipfile.ZipFile(zip_path, 'r') as z:
                for csv_name in z.namelist():
                    if not csv_name.lower().endswith('.csv'): continue
```

謝謝大家

